This documentation outlines a secure, scalable, and user-friendly Payment Tracking System designed for the Ethiopian market, leveraging Next.js and MongoDB.

1. System Architecture Overview

The system follows a modern, server-side rendered approach using Next.js (App Router), which allows for secure, fast data fetching and robust middleware protection.

- **Frontend:** Next.js with Tailwind CSS for styling and **Framer Motion** for smooth, modern UI animations.
- **Backend/API:** Next.js Server Actions and Route Handlers.
- **Database:** MongoDB (via Mongoose ODM) for flexible, document-based data storage.
- **Authentication: Auth.js (formerly NextAuth.js)** for secure session management, JWT handling, and role-based access control.

2. Core Features & Functional Modules

A. User Management (Admin-Only Registration)

- **Admin-Controlled:** Users cannot self-register. The Admin dashboard provides an "Add User" interface where the admin creates accounts using only a `username` and `email/phone`.
- **Account Security:** Users receive a temporary password or a secure activation link. Upon first login, the system forces a password change.
- **Management:** Admin can CRUD (Create, Read, Update, Delete) users and reset passwords if needed.

B. Payment Tracking & Integration

- **Local Bank Integration:** Integrate with Ethiopian Payment Service Providers (PSPs) like **AddisPay**, **Axra**, or bank-specific APIs (e.g., **CBE, Dashen, Bank of Abyssinia**).
- **Verification Flow:**
  1. User makes a payment through the chosen bank channel.
  2. The PSP sends a **Webhook** to your system upon transaction success.
  3. Your system validates the payload (using cryptographic signatures), updates the MongoDB payment status to "Paid", and automatically triggers an **Immediate Admin Notification**.

C. Smart Notifications & Filtering

- **Due-Date Alerts:** A server-side CRON job runs daily to query the database.
  - *Logic:* Find users where `next_payment_date` - `current_date` == 5 days. Send an automated "Payment Due" email/SMS.
- **Filtering:** Dashboard "At-a-glance" view showing:

- `Paid`: Filtered by month.
- `Pending/Overdue`: Filtered by month for proactive follow-up.

3. Database Schema Design (MongoDB)

Collection	Key Fields
Users	`_id`, `username`, `email/phone`, `passwordHash`, `role` (admin/user), `createdAt`
Payments	`_id`, `userId`, `amount`, `status` (pending/paid), `datePaid`, `paymentMonth`, `referenceNumber`
Notifications	`_id`, `userId`, `message`, `type` (reminder/alert), `isRead`, `createdAt`

4. Security Implementation

- **Middleware Protection:** Use Next.js Middleware to ensure only authorized users access the dashboard, and only Admins access registration routes.
- **Data Validation:** Use **Zod** to validate all incoming data from forms and API webhooks.
- **Encryption:** Passwords are hashed using `bcryptjs` before storage.
- **Secure Webhooks:** Validate the source of incoming payment notifications using the secret keys provided by the Ethiopian PSPs to prevent spoofing.

5. UI/UX & Modern Features

- **Dashboard:** Use **Shadcn/UI** components for a clean, professional aesthetic.
- **Animations:** Implement **Framer Motion** for:
  - Page transitions.
  - Success/Error toast notifications.
  - Loading skeletons while fetching payment status.
- **Mobile-First:** Ensure the interface is highly responsive, as most users in Ethiopia access services via mobile devices with varying bandwidths.

Suggested Implementation Roadmap

1. **Setup Auth:** Initialize Next.js and integrate Auth.js with MongoDB to handle admin and user sessions.
2. **Admin Portal:** Build the user management CRUD interface.
3. **Payment Sandbox:** Research the developer documentation for your chosen Ethiopian PSP (e.g., AddisPay) and test the webhook flow in their sandbox environment.
4. **Automation:** Implement a library like `node-cron` or a Vercel Cron function to handle the 5-day notification logic.

5. **Refine UI:** Apply Tailwind styling and Framer Motion components.